AETHERIS: A Distributed 2.5D Digital Environment Twin for Real-Time Microclimate Monitoring, Geospatial Telemetry and “What-If” Disaster Simulation

Devansh Joshi, Devansh Mittal, Dhruv Jain, Garv Kumar, Ishita Sinha

Department of Artificial Intelligence and Data Science, Arya College of Engineering & I.T., Jaipur, Rajasthan

Er. Ram Babu Buri is the Project Mentor and Guide (Assistant Professor, Department of Computer Science & Engineering).

E-mail: {devansh.joshi24ai, devansh.mittal24ai, dhruv.jain24ai, garv.kumar24ai, ishita.sinha24ai}@aryacollege.in, rambabu.buri@aryacollege.in

ABSTRACT

AETHERIS — Digital Environment Twin System is a web-based application designed to visualize environmental conditions and simplify the monitoring of air quality and microclimate data. It provides an interactive digital representation of environmental sensor stations and displays parameters such as Air Quality Index (AQI), temperature, humidity, particulate matter, and wind through a user-friendly dashboard. The application uses Java 17, Spring Boot 3.3.5, Spring Data JPA, MySQL, HTML5, CSS3, JavaScript, and HTML5 Canvas. Its features include user authentication, role-based access control, station management, GPS-based nearest-station identification, live and historical telemetry, and environmental alerts. It also provides a Clean Air Score (0–100), interactive 2.5D Digital Twin visualization, hazard simulations, automated telemetry updates, and environmental reports. The Haversine formula calculates the distance and bearing to the nearest registered station. AETHERIS is an academic prototype that primarily uses simulated environmental telemetry rather than guaranteed readings from physical sensors. The project demonstrates how web technologies, geospatial calculations, database management, and interactive visualization can make environmental information easy to understand.

Keywords: digital twin, environmental monitoring, Haversine formula, Spring Boot, 2.5D isometric rendering, disaster simulation, real-time telemetry.

LIST OF ABBREVIATIONS

Abbrev.	Expansion
API	Application Programming Interface
AQI	Air Quality Index
CORS	Cross-Origin Resource Sharing
DET	Digital Environment Twin
DTO	Data Transfer Object
EPA	Environmental Protection Agency
FPS	Frames Per Second
GIS	Geographic Information System
GPS	Global Positioning System
HiDPI	High Dots Per Inch
JPA	Java Persistence API
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
ORM	Object-Relational Mapping
PBL	Project-Based Learning
PM2.5/PM10	Particulate matter ≤ 2.5 µm / ≤ 10 µm
RBAC	Role-Based Access Control
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm, 256-bit
UI/UX	User Interface / User Experience

I. INTRODUCTION

Fine-grained, near-real-time observation of urban microclimates is increasingly important as cities grow and local heat and pollution events become more frequent. Observation practice is standardised by bodies such as the WMO [1], and air quality is communicated to the public through indices such as the US EPA Air Quality Index (AQI) [2]. In most deployments, however, sensor stations push readings to a central store that is then displayed as tables or time-series charts. We see three weaknesses in this pattern.

- No spatial context. Tables and line charts do not show how particulates disperse, how heat is distributed across a site, or where a station sits relative to the viewer.
- No safe rehearsal. Operators cannot test their response to a heatwave or an industrial release without risking real assets or waiting for a real event.
- Heavy client stacks. Interactive views are often built on large charting and 3D

libraries, which cost memory and load time on thin clients and low-end devices.

The digital twin paradigm addresses the first two points. A digital twin keeps a software counterpart synchronised with a physical process and uses it for monitoring and what-if analysis [3]–[6]. Here we present AETHERIS, a Digital Environment Twin built under the Project-Based Learning (PBL) curriculum at Arya College of Engineering & I.T., Jaipur. Its stations are simulated, as described in Section III-C, so AETHERIS is a complete twin platform ready to receive real sensor feeds rather than a deployment on physical hardware.

The main contributions are:

- a dependency-free 2.5D isometric Canvas engine with a particle system, three visualisation modes and a lightweight Bézier chart module;
- a geospatial pairing service that finds the nearest station and reports distance and compass direction from a browser GPS fix;
- a telemetry generator whose weighted update rule is analysed as a first-order mean-reverting process, with zone baselines;
- a role-guarded What-If simulation engine with threshold alerting and an acknowledgement lifecycle.

The rest of the paper is organised as follows. Section II describes the architecture and database. Section III gives the mathematical formulation. Section IV reports the evaluation. Section V discusses limitations, and Section VI concludes.

A. Team Responsibilities

The system was built by five students under the guidance of a faculty mentor. Devansh Joshi built the Haversine and bearing algorithms and the station and proximity APIs. Devansh Mittal built the telemetry generator, simulation engine and alert service. Dhruv Jain built the dashboard, KPI cards and location-pairing dialog. Garv Kumar designed the 3NF schema, the dual-database fallback, password hashing and RBAC. Ishita Sinha built the Canvas engine, particle system, dispersion halos and Bézier charts. Er. Ram Babu Buri is the project mentor and guide.

II. SYSTEM ARCHITECTURE AND PERSISTENCE

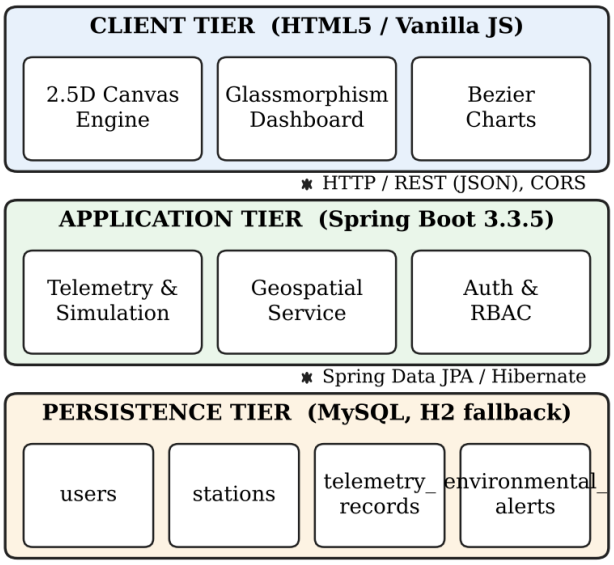


Fig. 1. Three-tier architecture of AETHERIS.

A. Multi-Tier Overview

AETHERIS separates presentation, application and persistence concerns (Fig. 1). The presentation tier uses semantic HTML5, ES6+ JavaScript and CSS3 with custom properties and `backdrop-filter: blur(12px)`; it has no framework dependency, which avoids a build step and keeps page load small [9]. A central controller polls the REST API every 5 seconds. The application tier runs on Java 17 and Spring Boot 3.3.5 [8], exposes REST controllers [7], and uses `@EnableScheduling` for background work [14]. The persistence tier uses Spring Data JPA with Hibernate. It connects to MySQL on port 3306 and, through `DataSourceConfig`, falls back to an in-memory H2 database if MySQL is unreachable, so the system starts with zero configuration. On startup a `DataInitializer` seeds six stations (NODE-01 to NODE-06), initial history and demonstration accounts.

B. Relational Model

The schema has four entities and is in third normal form [17]: `users` (credentials as salted hashes, role ADMIN/OPERATOR/RESEARCHER, last known coordinates); `stations` (identifier, name, state, latitude, longitude, elevation, status and zone type URBAN_CORE, INDUSTRIAL_PARK, FOREST_RESERVE or COASTAL_BASIN); `telemetry_records` (AQI, PM2.5, PM10, temperature, humidity, CO_2, noise, UV index, wind speed and direction,

and a status summary); and `environmental_alerts` (station, severity, category, description, acknowledged flag, timestamp). Stations relate 1:N to both telemetry records and alerts. A composite index on (`station_id`, `recorded_at`) supports the time-series queries used by the history endpoint.

III. MATHEMATICAL FORMULATION AND ENGINE DESIGN

A. Isometric Projection and Particle Physics

Station coordinates (φ, λ) are first normalised to the unit square, $X_n = (\lambda - \lambda_{min})/(\lambda_{max} - \lambda_{min})$ and $Y_n = (\varphi_{max} - \varphi)/(\varphi_{max} - \varphi_{min})$, scaled to the canvas, and then mapped to screen coordinates by the standard isometric transform with $\theta_{iso} = 30°$ (Fig. 2a):

$$X_s = (X_n - Y_n)\cdot\cos\theta_{iso} + X_{off} \quad (1)$$

$$Y_s = (X_n + Y_n)\cdot\sin\theta_{iso} + Y_{off} \quad (2)$$

A set of $N \geq 90$ particles represents wind-borne mist and particulates. Particle i has position p_i, velocity v_i, radius $r_i \in [1.0, 3.5]$ px and opacity $\alpha_i \in [0, 1]$. At each animation frame t:

$$p_i(t) = p_i(t-1) + v_i + w_{drift} \quad (3)$$

$$\alpha_i(t) = \alpha_i(t-1) - \delta_{decay} \quad (4)$$

where w_{drift} is the global wind vector. A particle that leaves the canvas re-enters on the opposite edge (Fig. 2b); one whose opacity reaches zero is re-initialised with a new random velocity, so the flow never empties. Because everything is plain 2D Canvas drawing, no shader compilation or mesh handling is needed. The canvas is scaled by `devicePixelRatio` for HiDPI screens and driven by `requestAnimationFrame` [9].

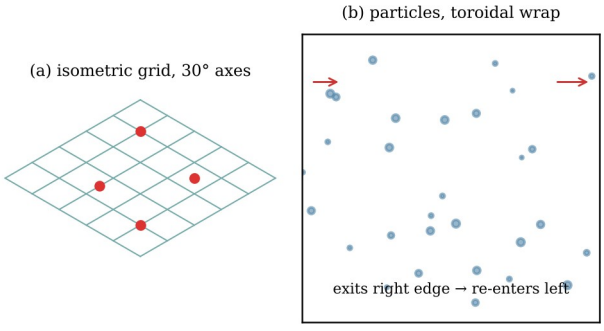


Fig. 2. (a) Isometric grid with projected station beacons; (b) particles re-entering at the opposite edge.

Three visualisation modes share this engine. Air dispersion draws a radial halo around each station whose radius and colour follow live AQI and PM2.5 on the EPA colour scale (green, yellow, orange/red, purple). Thermal draws heat contours from ambient temperature, graded from cyan to crimson. Radar draws a rotating sweep and sonar rings. Station selection uses Euclidean hit-testing, $\sqrt{(\Delta x^2 + \Delta y^2)} \leq 24$ px, and dispatches a `stationSelected` event to the dashboard. A separate module draws AQI and temperature trends with cubic Bézier splines and auto-scaled axes, without an external chart library.

B. Geospatial Pairing

When the user enables location, the browser Geolocation API [18] returns (φ_c, λ_c), which is sent to `GET /api/stations/nearest?lat=…&lng=…` (optionally filtered by `state`). The service evaluates the Haversine great-circle distance [10], [11] to every active station and returns the minimum:

$$a = \sin^2(\Delta\varphi/2) + k\cdot\sin^2(\Delta\lambda/2) \quad (5a)$$

$$d = 2R \cdot \arcsin(\sqrt{a}) \quad (5b)$$

with $k = \cos\varphi_c \cos\varphi_s$, $R = 6371$ km, and $\Delta\varphi$, $\Delta\lambda$ the station-minus-client differences in radians. Distances below 1 km are reported in metres (e.g. 450 m) and others in kilometres (e.g. 12.45 km). The initial bearing from client to station is

$$y = \sin\Delta\lambda \cdot \cos\varphi_s \quad (6a)$$

$$x = \cos\varphi_c \sin\varphi_s - \sin\varphi_c \cos\varphi_s \cos\Delta\lambda \quad (6b)$$

$$\theta = \text{atan2}(y, x) \quad (6c)$$

normalised by $\theta_{deg} = (\theta\cdot 180/\pi + 360) \bmod 360$, and mapped to one of 16 compass points by index $\lfloor(\theta_{deg} + 11.25)/22.5\rfloor \bmod 16$ into {N, NNE, NE, …, NNW}. The client then draws an animated geodesic beam from the user to the paired node.

C. Telemetry Generation

A scheduled task (`@Scheduled(fixedRate = 6000)`) produces one record per active station every 6 seconds. Each primary variable (AQI, temperature, humidity, CO_2, noise) is updated by

$$V_t = 0.85\, V_{(t-1)} + 0.15\, B_{zone} + \Delta_t \quad (7)$$

where B_{zone} is the baseline of the station's ecological zone (Table I) and Δ_t is uniform noise on (δ_{min}, δ_{max}). Eq. (7) is a first-order autoregressive, mean-reverting process, the discrete analogue of an Ornstein–Uhlenbeck process [13], rather than a pure random walk. Two properties follow directly. First, with zero-mean noise the expected value converges to B_{zone}.

Second, the memory coefficient a = 0.85 gives a relaxation time constant $\tau = -1/\ln a \approx 6.2$ ticks, about 37 s, so a change of baseline is absorbed smoothly instead of as a jump, and the stationary standard deviation is $\sigma_\Delta/\sqrt{(1 - a^2)} \approx 1.9\ \sigma_\Delta$. Dependent variables are derived from AQI: PM2.5 = round(0.48·AQI + ε_1, 1) and PM10 = round(1.80·PM2.5 + ε_2, 1), where ε_1 and ε_2 are small noise terms. UV index and wind are generated by the same scheme.

TABLE I. ZONE BASELINE PROFILES

Zone	AQI	T (°C)	RH (%)	CO_2 (ppm)	Noise (dB)
Urban core	140	30.0	48.0	560	68
Industrial	210	32.5	42.0	720	75
Forest	42	24.0	70.0	390	38
Coastal	88	26.5	74.0	420	48

D. What-If Simulation and Alerting

An operator with the ADMIN or OPERATOR role posts a scenario, an intensity k ∈ [0.5, 3.0] and a duration to `POST /api/simulation/trigger`. While a scenario is active its offsets and multipliers are applied to the baselines in Eq. (7), so the telemetry relaxes towards the disturbed state (Fig. 3). When the duration expires, or the `NORMAL` scenario is sent, the baselines are restored. Table II lists the scenarios (code names HEATWAVE, INDUSTRIAL_EMISSION, RAIN_CLEANSING, NORMAL).

TABLE II. SCENARIO DISTURBANCES (INTENSITY k)

Scenario	ΔT (°C)	ΔRH (%)	AQI ×	CO_2 ×
Heatwave	+7.5k	−15k	1.35k	—
Industrial emission	—	—	1.8k	1.6k
Rain cleansing	−4.5k	+22k	0.5/k	—
Normal	reset	reset	1	1

Within each heartbeat the new state is checked against two thresholds taken from the EPA scale [2]: AQI ≥ 250 raises a CRITICAL “Severe Air Quality Hazard” alert, and temperature ≥ 38.0 °C raises a HAZARD “Extreme Thermal Heat Spike” alert. To avoid writing a new alert every 6 seconds, no further alert of the same category is stored for a station while one remains unacknowledged. Operators list open alerts with `GET /api/alerts?unacknowledgedOnly=true` and close them with `POST /api/alerts/{id}/acknowledge`.

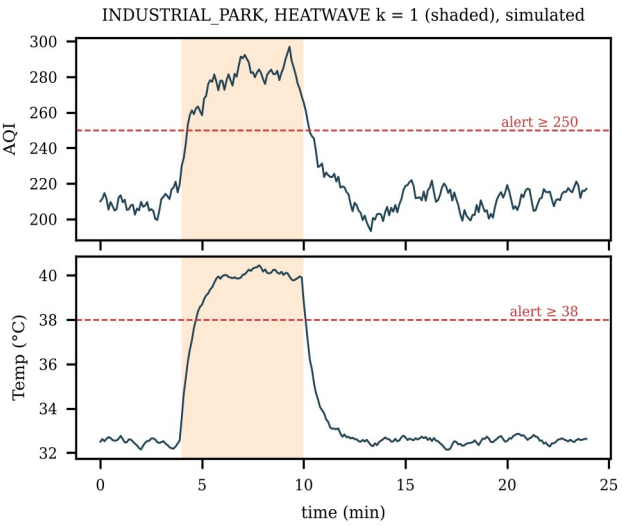


Fig. 3. Simulated response of an industrial-park node to a HEATWAVE (k = 1, shaded). Dashed lines mark the alert thresholds. Noise U(−6, 6) for AQI and U(−0.15, 0.15) °C for temperature; illustrative parameters.

Because the expected value of Eq. (7) tends to the disturbed baseline, the steady-state outcome of each scenario can be tabulated in advance (Table III). The table shows that only an industrial-zone heatwave or an industrial emission is expected to trip the alerts at k = 1, and that an emission at the urban-core baseline sits just above the AQI threshold (252).

TABLE III. EXPECTED STEADY STATE AT k = 1 (AQI / T °C); BOLD = ALERT

Zone	Baseline	Heatwave	Emission	Rain
Urban core	140 / 30.0	189 / 37.5	**252 / 30.0**	70 / 25.5
Industrial	210 / 32.5	**284 / 40.0**	**378 / 32.5**	105 / 28.0
Forest	42 / 24.0	57 / 31.5	76 / 24.0	21 / 19.5
Coastal	88 / 26.5	119 / 34.0	158 / 26.5	44 / 22.0

IV. EXPERIMENTAL EVALUATION

A. Rendering Performance

The Canvas engine was profiled with the Chrome DevTools Performance Monitor on three display profiles (Table IV). In all three the frame time stayed far below the 16.67 ms budget for 60 FPS, which is consistent with the absence of WebGL shader compilation and mesh transforms. The mobile profile used 60 particles instead of 95.

TABLE IV. CANVAS RENDERING PERFORMANCE

Metric	1080p desktop	Retina / HiDPI	Mobile / tablet
Mean frame rate (FPS)	60.0	59.8	59.9
Frame render time (ms)	2.14	3.82	4.10
Active particles	95	95	60
JS heap (MB)	18.4	21.2	16.8

B. Geospatial Accuracy

The Haversine implementation was checked against the WGS-84 ellipsoidal geodesic computed with Karney's algorithm [12] (the `Geod` routine of PROJ/pyproj). Three short urban pairs and one long pair (Delhi to Mumbai) were used (Table V). A sphere of radius 6371 km cannot match the ellipsoid exactly, so a difference of a few tenths of a percent is expected: the mean error was 0.28 % and the largest 0.42 %, equivalent to about 75 m over 17.6 km. This is adequate for choosing the nearest station and for display. The bearing agreed with the ellipsoidal azimuth to within 0.2° on every pair. Server-side handling of `/api/stations/nearest` averaged 0.84 ms.

TABLE V. HAVERSINE VERSUS WGS-84 GEODESIC

Pair	Haversine (km)	Geodesic (km)	Error (%)	Bearing
A	19.80	19.81	0.09	116.1° ESE
B	17.71	17.64	0.42	194.8° SSW
C	6.00	5.98	0.29	144.1° SE
D	1148.10	1144.53	0.31	203.5° SSW

A: 28.6139 N, 77.2090 E → 28.5355 N, 77.3910 E. B: 19.0760, 72.8777 → 18.9220, 72.8347. C: 12.9716, 77.5946 → 12.9279, 77.6271. D: Delhi 28.6139, 77.2090 → Mumbai 19.0760, 72.8777.

C. Scheduler Stability

The backend was run continuously for 24 hours on an Intel Core i7 machine with Java 17 and six stations. The 6-second heartbeat produced 14,400 ticks and therefore 86,400 telemetry records (6 × 14,400). No crash or thread starvation occurred. The heap after garbage collection was 42 MB at the start and 45 MB at hour 24, so growth was small and bounded. A paginated history query (`PageRequest.of(0, 25)`) took 14.2 ms on average, helped by the composite index on (`station_id`, `recorded_at`).

V. DISCUSSION AND LIMITATIONS

- Simulated stations. All telemetry is generated, so the evaluation measures the software, not environmental accuracy. Validation against real sensors remains future work.
- Small test scale. Six stations and one test machine were used; scalability beyond that is not shown.
- Password hashing. Salted SHA-256 [15] avoids plain-text storage, but a fast hash is weaker against offline guessing than a deliberately slow, memory-hard function such as bcrypt or Argon2 [16]; this should be changed before real deployment.
- Polling. The client polls every 5 s while the server ticks every 6 s, so a view can lag by up to one interval; push updates would remove this.
- Scenario model. The multipliers in Table II are engineering choices, not calibrated atmospheric models.

VI. CONCLUSION AND FUTURE WORK

AETHERIS shows that a useful digital environment twin can be built with a small, framework-free browser client and a conventional Spring Boot and MySQL backend. It combines an isometric Canvas view, accurate nearest-station pairing, smooth mean-reverting telemetry and an operator-driven What-If engine with automated alerts. Future work includes connecting real ESP32 or LoRaWAN sensors through an MQTT bridge, predicting plume dispersion with LSTM networks, replacing polling with WebSockets, and moving to a slow password hash.

ACKNOWLEDGEMENT

The authors thank Er. Ram Babu Buri, Assistant Professor, Department of Computer Science & Engineering, Arya College of Engineering & I.T., for guidance and mentorship throughout this PBL project.

REFERENCES

[1] World Meteorological Organization, Guide to Instruments and Methods of Observation, WMO-No. 8, Geneva, 2018.

[2] U.S. EPA, Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI), EPA-454/B-18-007, Sept. 2018.

[3] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in Transdisciplinary Perspectives on Complex Systems, Springer, 2017, pp. 85–113.

[4] F. Tao, H. Zhang, A. Liu and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[5] A. Fuller, Z. Fan, C. Day and C. Barlow, “Digital twin: Enabling technologies, challenges and open research,” IEEE Access, vol. 8, pp. 108952–108971, 2020.

[6] A. Rasheed, O. San and T. Kvamsdal, “Digital twin: Values, challenges and enablers from a modeling perspective,” IEEE Access, vol. 8, pp. 21980–22012, 2020.

[7] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, 2000.

[8] C. Walls, Spring Boot in Action, Manning, 2016.

[9] Mozilla Developer Network, “Optimizing canvas,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API/Tutorial/Optimizing_canvas

[10] R. W. Sinnott, “Virtues of the haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[11] C. Veness, “Calculate distance, bearing and more between latitude/longitude points,” Movable Type Scripts. [Online]. Available: https://www.movable-type.co.uk/scripts/latlong.html

[12] C. F. F. Karney, “Algorithms for geodesics,” J. Geodesy, vol. 87, no. 1, pp. 43–55, 2013.

[13] G. E. Uhlenbeck and L. S. Ornstein, “On the theory of the Brownian motion,” Phys. Rev., vol. 36, no. 5, pp. 823–841, 1930.

[14] Spring Framework Documentation, “Task execution and scheduling.” [Online]. Available: https://docs.spring.io/spring-framework/reference/integration/scheduling.html

[15] NIST, Secure Hash Standard (SHS), FIPS PUB 180-4, Aug. 2015.

[16] OWASP, “Password storage cheat sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[17] E. F. Codd, “Further normalization of the data base relational model,” in Data Base Systems, Courant Computer Science Symposia 6, Prentice-Hall, 1972, pp. 33–64.

[18] W3C, “Geolocation API,” W3C Recommendation. [Online]. Available: https://www.w3.org/TR/geolocation/